

Программирование(одинаковая для всех разработчиков)

Цель интервью

- Проверить навыки кандидата в написании и отлаживании (при необходимости) кода.
- Проверить умение кандидата обеспечить работу решения не только для предоставленных примеров (корнер-кейсы)
- Проверить знания кандидата базовых структур данных (список, стек, хэш-таблица и тд) и алгоритмов (например, рекурсивный, линейный и тд)
- Проверить знание и понимание кандидатом алгоритмической сложности

В результате этого интервью определяем уровень кандидата: ниже начального, начальный, средний или продвинутый.

Также обращаем внимание на то, как кандидат общается (отмечаем в отчете агрессию в сторону формата вопросов/интервью, мат и тд)

Как определить уровень кандидата

Ниже начального

- Не решил ни одной задачи
- Не смог самостоятельно придумать алгоритм для решения

Начальный

- Решил easy-задачу
- Придумал алгоритм по easy задаче и перенес его в код. Подсказки возможны (набор данных, на которых сломается решение)
- Может посчитать алгоритмическую сложность своего решения (хотя бы на уровне "цикл в цикле – это $O(N^2)$ ")
- Знает алгоритмическую сложность операций с распространенными структурами данных (динамический массив, хеш-таблица)
- Может отладить написанное в уме, в ходе дебага находит ошибки
- Способен придумать корнер кейсы

Средний

- Решил easy и medium задачу

- Придумал алгоритм по easy задаче и перенес его в код. Приемлемы подсказки, не влияющие на итоговый алгоритм (например, наводящие вопросы про работу со структурами: как найти элемент в списке? как быстро это будет работать?)
- Придумал алгоритм по medium задаче и перенес его в код. Приемлемы подсказки, не влияющие на итоговый алгоритм
- Оценивает алгоритмическую сложность. Правильно назвал сложность алгоритма. Правильно определил потребление памяти
- Знает алгоритмическую сложность операций с распространенными структурами данных (динамический массив, хеш-таблица)
- Способен придумать кейсы

Продвинутый

- То же самое, что и для среднего
- Решил все задачи оптимально с точки зрения временной/пространственной сложности
- Задачи не требуют сильного рефакторинга
- Решил все задачи без подсказок
- Пояснял свои решения, рассуждал в процессе
- Свободно оценивает алгоритмическую сложность
 - Знает сложности основных сортировок (quick, merge, bubble)
 - Различает сложность в худшем/лучшем/среднем случае
 - Понимает про константы и доминанты

До интервью

- [Создаем](#) новый отчет;
- Заполняем поля «Кандидат», «Интервьюер(ы)» и «Дата».

Как начать интервью

«Привет, меня зовут %ИМЯ%, я работаю в Авито в %отделе подсчета денег%.
Ничего если я буду на ты?

Я проведу у тебя секцию Программирования. Она рассчитана на час времени, в течение которого мы решим одну или несколько задач. Тебе нужно написать правильное решение, в конце задачи оценить сложность по времени и по памяти.

Будет хорошо, если во время решения ты будешь размышлять вслух и комментировать свои шаги. От тебя не требуется точно знать названия функций

или синтаксис языка. Но если ты используешь что-то из стандартной библиотеки, типа сортировки, должен объяснить допущения по сложности.

Сейчас я скину ссылку на блокнотик avito.code, где мы будем писать код.»

Во время интервью

Предлагаем решить несколько задач из [библиотеки задач](#), учитывая уровень кандидата и требования к вакансии (одна задача уровня **easy** и одна задача **medium**). Если кандидат использует стандартную библиотеку, готовые структуры данных и т.п, то он должен уметь объяснить, как оно работает внутри "на пальцах".

Оценка решения кандидата

Рекомендации:

- Не требуем изначально максимально эффективного решения. Сначала надо просто решить задачу. Далее можно оценить текущее решение и подвести кандидата к поиску более оптимального.
- Обращаем внимание на:
 - Ясность мышления / умение объяснить, как работает написанный код
 - Наименование переменных / функций (понятное, не слишком длинное, не слишком короткое)
 - Форматирование кода
- НЕ обращаем внимание на:
 - Точки с запятыми
 - Единичные опечатки
 - Модные фишки языка
 - Ошибки при рефакторинге (где-то что-то забыл переименовать)
 - Если забыл(а) имя функции из стандартной библиотеки - подсказываем или выдумываем

Артефактом решения задач должен стать полный текст задач в файле отчета и решения, предложенные кандидатом, а также ответы кандидата на дополнительные вопросы.

Как закончить интервью

«У меня вопросов больше нет. Может, ты хочешь что-нибудь спросить у меня? Если нет, то я могу рассказать, как у нас всё устроено в Авито или что будет дальше на следующих этапах.»

Отвечаем на вопросы, объясняем остальные секции и примерные сроки; попутно рассказываем, за что вы сами любите Авито – это помогает продать вакансию

«Спасибо за то, что пришел(ла), было приятно с тобой попрограммировать, удачи на следующих этапах.»

После интервью

- «Причесываем» заметки по ответам кандидата;
- Заполняем поля «Ссылка на код», «Оценка уровня» и оставляем комментарий на основе чего было принято это решение;
- Нажимаем «Сохранять отчет»;
- Переходим на детальную страницу интервью по ссылке в календаре (приглашении), добавляем ссылку на отчет в поле «Ссылка на итоги», выставляем «Уровень» и завершаем интервью.

Инструменты

- Для ведения [протокола](#)
- Для онлайн-кодинга с [подсветкой](#) или [исполнением](#)
- Онлайн whiteboard: [Draw.io](#), [Miro](#), [Whimsical](#) или [Excalidraw](#)

Приведенные ниже задачи являются шаблонными на текущий период для решения на секции. Задачи проходят ротацию раз в квартал.

*Полный [список базовых задач](#), из которого собеседующий по желанию может выбрать другие задачи соответствующей сложности. (Должны даны быть две задачи: *easy* и *medium*)*

Задачи

Условие задачи

Мы в Авито любим проводить соревнования, — недавно мы устроили чемпионат по шагам. И вот настало время подводить итоги!

Необходимо определить userIds участников, которые прошли наибольшее количество шагов steps за все дни, не пропустив ни одного дня соревнований.

Пример

```

# Пример 1
# ввод
statistics = [
    [{ userId: 1, steps: 1000 }, { userId: 2, steps: 1500 }],
    [{ userId: 2, steps: 1000 }]
]

# вывод
champions = { userIds: [2], steps: 2500 }

# Пример 2
statistics = [
    [{ userId: 1, steps: 2000 }, { userId: 2, steps: 1500 }],
    [{ userId: 2, steps: 4000 }, { userId: 1, steps: 3500 }]
]

# вывод
champions = { userIds: [1, 2], steps: 5500 }

```

Алгоритмическая сложность

Т.к. в условии не оговорено ограничение на массив statistics, правильными являются оба ответа (главное, чтобы кандидат объяснил, что он подразумевает под N и K в каждом случае). Ожидаемая сложность: $O(NK)$, где N - количество дней(строк в массиве statistics), K - максимальное количество участников в день. или $O(N)$, где N - суммарное количество записей в массиве за все дни.

Способы решения

Проход по всем элементам с записью в карту, затем фильтрация карты по условиям задачи.

Возможные вопросы

Спрашивать об оптимизациях, если в карту кладутся абсолютно все участники, не принимавшие участия в предыдущих днях.

Например, если в первый день участвовал 1 человек, а во второй 10 000, то нет смысла класть их всех в карту.

Основные ошибки/корнер-кейсы

Может быть несколько победителей с одинаковым количеством шагов. Победителей может не быть вообще(если каждый участник участвовал не во все дни). На каждом шаге проверять, что участник был в предыдущем через includes(неоптимально). Использование сортировки(неоптимально).

Пример решения

Golang

Если кандадат пишет на go, можно дать ему такой шаблон, чтобы не мучался с мапками

```
package main

import (
    "fmt"
    "reflect"
)

type Statistic struct {
    UserID int
    Steps  int
}

type Result struct {
    UserIDs []int
    Steps   int
}

func getChampions(statistics [][]Statistic) Result {
    // Реализуй функцию
}

func main() {
    fmt.Println("example 1", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 2000}, {UserID: 2, Steps: 1500}},
                {{UserID: 2, Steps: 1000}},
            },
        ),
        Result{
            UserIDs: []int{2},
            Steps:   2500,
        },
    ))
    fmt.Println("example 2", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 1000}, {UserID: 2, Steps: 1500}, {UserID: 3, Steps: 1500}},
                {{UserID: 2, Steps: 1000}, {UserID: 1, Steps: 1000}, {UserID: 3, Steps: 1000}},
            },
        ),
        Result{
            UserIDs: []int{2, 3},
            Steps:   2500,
        },
    ))
}
```

```

    },
  ))
}

```

Решение + тесты

```

package main

import (
    "fmt"
    "reflect"
    "sort"
)

type Statistic struct {
    UserID int
    Steps  int
}

type Result struct {
    UserIDs []int
    Steps   int
}

func getChampions(statistics [][]Statistic) Result {
    if len(statistics) == 0 {
        return Result{}
    }

    type UserStatistic struct {
        DaysIn  int
        StepsSum int
    }

    possibleChampions := make(map[int]UserStatistic, len(statistics[0]))
    for _, userStat := range statistics[0] {
        possibleChampions[userStat.UserID] = UserStatistic{
            DaysIn:  1,
            StepsSum: userStat.Steps,
        }
    }

    for day := 1; day < len(statistics); day++ {
        for _, userDayStatistic := range statistics[day] {
            userID := userDayStatistic.UserID
            if dayStatistic, found := possibleChampions[userID]; found {
                dayStatistic.DaysIn++
                dayStatistic.StepsSum += userDayStatistic.Steps
                possibleChampions[userID] = dayStatistic
            }
        }
    }
}

```

```

    }
}

maxSteps := -1
for _, stat := range possibleChampions {
    if stat.DaysIn == len(statistics) {
        maxSteps = max(maxSteps, stat.StepsSum)
    }
}

winners := make([]int, 0, len(possibleChampions))
for userID, stat := range possibleChampions {
    if stat.StepsSum == maxSteps && stat.DaysIn == len(statistics) {
        winners = append(winners, userID)
    }
}
if len(winners) == 0 {
    return Result{}
}

return Result{
    UserIDs: winners,
    Steps:   maxSteps,
}
}

func main() {
    fmt.Println("empty", reflect.DeepEqual(getChampions(nil), Result{}))
    fmt.Println("case 1", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 2000}, {UserID: 2, Steps: 1500}},
                {{UserID: 2, Steps: 1000}},
            },
        ),
        Result{
            UserIDs: []int{2},
            Steps:   2500,
        },
    ))
    fmt.Println("case 2", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 2000}, {UserID: 2, Steps: 1500}},
                {{UserID: 2, Steps: 1000}, {UserID: 3, Steps: 4000}},
            },
        ),
        Result{
            UserIDs: []int{2},
            Steps:   2500,
        },
    ))
}

```



```

    },
    ))
    fmt.Println("case 3", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 2000}, {UserID: 2, Steps: 1500}},
                {{UserID: 2, Steps: 1000}, {UserID: 1, Steps: 3000}},
            },
        ),
        Result{
            UserIDs: []int{1},
            Steps: 5000,
        },
    ))
    // А тут у нас может быть флакующий тест, и так как мы ничего не импортируем нормального, то
    напомним так:
    case4Answer := getChampions(
        [][]Statistic{
            {{UserID: 1, Steps: 1000}, {UserID: 2, Steps: 1500}, {UserID: 3, Steps: 1500}},
            {{UserID: 2, Steps: 1000}, {UserID: 1, Steps: 1000}, {UserID: 3, Steps: 1000}},
        },
    )
    sort.Ints(case4Answer.UserIDs)
    fmt.Println("case 4", reflect.DeepEqual(
        case4Answer,
        Result{
            UserIDs: []int{2, 3},
            Steps: 2500,
        },
    ))
    fmt.Println("case 5", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 2000}, {UserID: 2, Steps: 1500}},
                {{UserID: 2, Steps: 1000}},
                {{UserID: 3, Steps: 1000}},
            },
        ),
        Result{},
    ))
    fmt.Println("case 6", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 5000}, {UserID: 2, Steps: 1500}},
                {{UserID: 2, Steps: 1000}},
            },
        ),
        Result{
            UserIDs: []int{2},
            Steps: 2500,
        },
    ))

```

```
    ))  
  }
```

JavaScript

Решение

```
/**  
 *  
 * @param {Array<Array<{userId: number; steps: number;}>>} statistics  
 * @returns {{userIds: number[]; steps: number;} | null}  
 */  
function getChampions(statistics = []) {  
  // Только те, кто был с самого начала, могут победить  
  const possibleChampions = statistics[0].reduce((acc, next) => {  
    acc[next.userId] = {  
      daysIn: 1,  
      steps: next.steps  
    };  
    return acc;  
  }, {});  
  
  for (let i = 1; i < statistics.length; i++) {  
    for (const result of statistics[i]) {  
      if (possibleChampions.hasOwnProperty(result.userId) && possibleChampions[result.userId].daysIn  
=== i) {  
        possibleChampions[result.userId].steps += result.steps;  
        possibleChampions[result.userId].daysIn += 1;  
      }  
    }  
  }  
  
  // Берем только тех, кто каждый день участвовал  
  const activeChampions = Object.entries(possibleChampions).filter(([_, { daysIn }]) => daysIn ===  
statistics.length);  
  const maxSteps = Math.max(...activeChampions.map(([_, { steps }]) => steps));  
  
  if (maxSteps === Number.NEGATIVE_INFINITY) {  
    return null;  
  }  
  
  const winners = {  
    userIds: [],  
    steps: maxSteps  
  };  
  
  for (let champion of activeChampions) {  
    const [userId, { steps }] = champion  
    if (steps === maxSteps) {  
      winners.userIds.push(+userId);  
    }  
  }  
}
```

```
}  
}
```

Тесты

```
console.log(getChampions(  
  [  
    [{ userId: 1, steps: 2000 }, { userId: 2, steps: 1500 }],  
    [{ userId: 2, steps: 1000 }]  
  ]  
));  
  
// expected: { userIds: [2], steps: 2500 }
```

```
console.log(getChampions(  
  [  
    [{ userId: 1, steps: 2000 }, { userId: 2, steps: 1500 }],  
    [{ userId: 2, steps: 1000 }, { userId: 3, steps: 4000 }]  
  ]  
));  
  
// expected: { userIds: [2], steps: 2500 }
```

```
console.log(getChampions(  
  [  
    [{ userId: 1, steps: 2000 }, { userId: 2, steps: 1500 }],  
    [{ userId: 2, steps: 1000 }, { userId: 1, steps: 3000 }]  
  ]  
));  
  
// expected: { userIds: [1], steps: 5000 }
```

```
console.log(getChampions(  
  [  
    [{ userId: 1, steps: 500 }, { userId: 2, steps: 1500 }, { userId: 3, steps: 1000 }],  
    [{ userId: 2, steps: 1000 }, { userId: 1, steps: 2000 }, { userId: 3, steps: 1000 }]  
  ]  
));  
  
// expected: { userIds: [1, 2], steps: 2500 }
```

```
console.log(getChampions(  
  [  
    [{ userId: 1, steps: 500 }, { userId: 2, steps: 1500 }, { userId: 3, steps: 1000 }, { userId: 4, steps: 2500 }, {  
userId: 5, steps: 12500 }],  
    [{ userId: 2, steps: 1000 }, { userId: 1, steps: 2000 }, { userId: 3, steps: 1000 }]  
  ]  
));  
// expected: { userIds: [1, 2], steps: 2500 }
```

```
console.log(getChampions(  
  [  
    [{ userId: 1, steps: 500 }, { userId: 2, steps: 1500 }],  
    [{ userId: 2, steps: 1000 }, { userId: 3, steps: 1000 }],  
    [{ userId: 3, steps: 1000 }]  
  ]  
));  
  
// expected: null
```

Kotlin

Решение

```
data class UserStatistics(val steps: Int = 0, val amountDay: Int = 0)
class DayStatistics(val userId: Int, val steps: Int)
class Winners(val userIds: IntArray = intArrayOf(), val steps: Int = 0)

fun getChampions(statistics: Array<Array<DayStatistics>>): Winners {
    if (statistics.isEmpty()) {
        return Winners()
    }
    val map = HashMap<Int, UserStatistics>()
    for (i in statistics.indices) { //аһалор i >= 0 && i < statistics.size
        for (j in statistics[i].indices) {

            val dayStatistics = statistics[i][j]
            if (map[dayStatistics.userId] == null && i == 0) {
                map[dayStatistics.userId] = UserStatistics()
            }
            map[dayStatistics.userId]?.let {
                if (it.amountDay != i) {
                    map.remove(dayStatistics.userId)
                } else {
                    it.steps += dayStatistics.steps
                    it.amountDay++
                }
            }
        }
    }
}

var maxSteps = 0
var resUserIds = ArrayList<Int>()
for (user in map.entries) {
    if (user.value.amountDay == statistics.size) {
        if (maxSteps == user.value.steps) {
            resUserIds.add(user.key)
        } else if (maxSteps < user.value.steps) {
            resUserIds = arrayListOf(user.key)
            maxSteps = user.value.steps
        }
    }
}
```

```

    }
}

return Winners(resUserIds.toIntArray(), maxSteps)
}

```

Тесты

```

fun test(message: String, input: Array<Array<DayStatistics>>, output: Winners) {
    val result = getChampions(input)
    println(
        "$message, ${
            output.userIds.contentEquals(result.userIds) && output.steps == result.steps
        }"
    )
}

```

```

test(
    "sample1", arrayOf(
        arrayOf(
            DayStatistics(1, 2000),
            DayStatistics(2, 1500)
        ),
        arrayOf(DayStatistics(2, 1000))
    ), Winners(intArrayOf(2), 2500)
)

```

```

test(
    "sample2", arrayOf(
        arrayOf(
            DayStatistics(1, 2000),
            DayStatistics(2, 1500)
        ),
        arrayOf(
            DayStatistics(2, 1000),
            DayStatistics(3, 4000)
        )
    ), Winners(intArrayOf(2), 2500)
)

```

```

test(
    "sample3", arrayOf(
        arrayOf(
            DayStatistics(1, 2000),
            DayStatistics(2, 1500)
        ),
        arrayOf(
            DayStatistics(2, 1000),
            DayStatistics(1, 3000)
        )
    )
)

```

```

    ), Winners(intArrayOf(1), 5000)
)

test(
    "sample4", arrayOf(
        arrayOf(
            DayStatistics(1, 500),
            DayStatistics(2, 1500),
            DayStatistics(3, 1000)
        ),
        arrayOf(
            DayStatistics(2, 1000),
            DayStatistics(1, 2000),
            DayStatistics(3, 1000)
        )
    ), Winners(intArrayOf(1, 2), 2500)
)

test(
    "sample5", arrayOf(
        arrayOf(
            DayStatistics(1, 500),
            DayStatistics(2, 1500)
        ),
        arrayOf(
            DayStatistics(2, 1000),
            DayStatistics(3, 1000)
        ),
        arrayOf(
            DayStatistics(3, 1000)
        )
    ), Winners(intArrayOf(), 0)
)

```

Swift

Решение

```

struct UserStat {
    let userId: Int
    let steps: Int
}

typealias DailyStats = [UserStat]

struct Champions {
    let userIds: [Int]
    let totalSteps: Int
}

func getChampions(_ stats: [DailyStats]) -> Champions? {

```

```

struct Competitor {
    var totalSteps: Int
    var daysIn: Int
}

var daysPassed = 1
var competitors = [Int: Competitor]()

stats.first?.forEach { userStat in
    competitors[userStat.userId] = Competitor(
        totalSteps: userStat.steps,
        daysIn: daysPassed
    )
}

guard !competitors.isEmpty
else { return nil }

for dayStat in stats.dropFirst() {
    daysPassed += 1
    for userStat in dayStat {
        if let competitor = competitors[userStat.userId] {
            competitors[userStat.userId] = Competitor(
                totalSteps: competitor.totalSteps + userStat.steps,
                daysIn: daysPassed
            )
        }
    }
}

// Filtering out competitors who missed the day
competitors = competitors.filter { competitor in
    competitor.value.daysIn == daysPassed
}

guard !competitors.isEmpty
else { return nil }

var maxSteps = 0
var winners = [Int]()

for competitor in competitors {
    if competitor.value.totalSteps > maxSteps {
        winners = [competitor.key]
        maxSteps = competitor.value.totalSteps
    } else if competitor.value.totalSteps == maxSteps {
        winners.append(competitor.key)
    }
}

```

```

    return Champions(userIds: winners.sorted(), totalSteps: maxSteps)
}

```

Решение с использованием редьюсеров

```

struct UserStat {
    let userId: Int
    let steps: Int
}

```

```

typealias DailyStats = [UserStat]

```

```

struct Champions {
    let userIds: [Int]
    let totalSteps: Int
}

```

```

func getChampions(_ stats: [DailyStats]) -> Champions? {
    struct Competitor {
        var totalSteps: Int
        var daysIn: Int
    }

```

```

    let firstDayCompetitors = stats.first?.reduce(into: [Int: Competitor]() { competitors, userStat in
        competitors[userStat.userId] = Competitor(
            totalSteps: userStat.steps,
            daysIn: 1
        )
    })
}

```

```

guard let firstDayCompetitors, !firstDayCompetitors.isEmpty
else { return nil }

```

```

let competitors = stats.dropFirst().reduce(into: firstDayCompetitors) { competitors, dayStat in
    dayStat.forEach { userStat in
        if let competitor = competitors[userStat.userId] {
            competitors[userStat.userId] = Competitor(
                totalSteps: competitor.totalSteps + userStat.steps,
                daysIn: competitor.daysIn + 1
            )
        }
    }
}
}

```

```

// Filtering out competitors who missed some days
let totalDays = stats.count
let allDaysCompetitors = competitors.filter { competitor in
    competitor.value.daysIn == totalDays
}

```

```

guard !allDaysCompetitors.isEmpty

```



```

    else { return nil }

    let maxSteps = allDaysCompetitors.reduce(-1) { maxSteps, competitor in
        max(maxSteps, competitor.value.totalSteps)
    }

    let winners = allDaysCompetitors.compactMap { userId, competitor in
        competitor.totalSteps == maxSteps ? userId : nil
    }

    return Champions(userIds: winners.sorted(), totalSteps: maxSteps)
}

```

Тесты

```

extension Champions: Equatable {}

func test(_ message: String, _ input: [DailyStats], _ output: Champions?) {
    let success = getChampions(input) == output
    print("\(message) > \(success ? "success" : "failed")")
}

test("sample1", [],
    nil
)
test("sample2", [
    [UserStat(userId: 1, steps: 2000), UserStat(userId: 2, steps: 1500)],
    [UserStat(userId: 2, steps: 1000)]
],
    Champions(userIds: [2], totalSteps: 2500)
)
test("sample3", [
    [UserStat(userId: 1, steps: 2000), UserStat(userId: 2, steps: 1500)],
    [UserStat(userId: 2, steps: 1000), UserStat(userId: 3, steps: 4000)]
],
    Champions(userIds: [2], totalSteps: 2500)
)
test("sample4", [
    [UserStat(userId: 1, steps: 2000), UserStat(userId: 2, steps: 1500)],
    [UserStat(userId: 2, steps: 1000), UserStat(userId: 1, steps: 3000)]
],
    Champions(userIds: [1], totalSteps: 5000)
)
test("sample5", [
    [UserStat(userId: 1, steps: 500), UserStat(userId: 2, steps: 1500), UserStat(userId: 3, steps: 1000)],
    [UserStat(userId: 2, steps: 1000), UserStat(userId: 1, steps: 2000), UserStat(userId: 3, steps: 1000)]
],
    Champions(userIds: [1,2], totalSteps: 2500)
)
test("sample6", [
    [UserStat(userId: 1, steps: 500), UserStat(userId: 2, steps: 1500)],

```

```
[UserStat(userId: 2, steps: 1000), UserStat(userId: 3, steps: 1000)],  
[UserStat(userId: 3, steps: 1000)]  
],  
nil  
)
```

Задача 2:

Условие задачи

Мы в Авито любим проводить соревнования, — недавно мы устроили чемпионат по шагам. И вот настало время подводить итоги!

Необходимо определить userIds участников, которые прошли наибольшее количество шагов steps за все дни, не пропустив ни одного дня соревнований.

Пример

```
# Пример 1  
# ВВОД  
statistics = [  
  [{ userId: 1, steps: 1000 }, { userId: 2, steps: 1500 }],  
  [{ userId: 2, steps: 1000 }]  
]
```

```
# ВЫВОД  
champions = { userIds: [2], steps: 2500 }
```

```
# Пример 2  
statistics = [  
  [{ userId: 1, steps: 2000 }, { userId: 2, steps: 1500 }],  
  [{ userId: 2, steps: 4000 }, { userId: 1, steps: 3500 }]  
]
```

```
# ВЫВОД  
champions = { userIds: [1, 2], steps: 5500 }
```

Алгоритмическая сложность

Т.к. в условии не оговорено ограничение на массив statistics, правильными являются оба ответа (главное, чтобы кандидат объяснил, что он подразумевает под N и K в каждом случае). Ожидаемая сложность: * $O(NK)$, где N - количество дней(строк в массиве statistics), K - максимальное количество участников в день. или * $O(N)$, где N - суммарное количество записей в массиве за все дни.

Способы решения

Проход по всем элементам с записью в карту, затем фильтрация карты по условиям задачи.

Возможные вопросы

Спрашивать об оптимизациях, если в карту кладутся абсолютно все участники, не принимавшие участия в предыдущих днях.

Например, если в первый день участвовал 1 человек, а во второй 10 000, то нет смысла класть их всех в карту.

Основные ошибки/корнер-кейсы

Может быть несколько победителей с одинаковым количеством шагов. Победителей может не быть вообще(если каждый участник участвовал не во все дни). На каждом шаге проверять, что участник был в предыдущем через `includes`(неоптимально). Использование сортировки(неоптимально).

Пример решения

Golang

Если кандидат пишет на go, можно дать ему такой шаблон, чтобы не мучался с мапками

```
package main

import (
    "fmt"
    "reflect"
)

type Statistic struct {
    UserID int
    Steps int
}

type Result struct {
    UserIDs []int
    Steps int
}

func getChampions(statistics [][]Statistic) Result {
```

```

    // Реализуй функцию
}

func main() {
    fmt.Println("example 1", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 2000}, {UserID: 2, Steps: 1500}},
                {{UserID: 2, Steps: 1000}},
            },
        ),
        Result{
            UserIDs: []int{2},
            Steps: 2500,
        },
    ))
    fmt.Println("example 2", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 1000}, {UserID: 2, Steps: 1500}, {UserID: 3, Steps: 1500}},
                {{UserID: 2, Steps: 1000}, {UserID: 1, Steps: 1000}, {UserID: 3, Steps: 1000}},
            },
        ),
        Result{
            UserIDs: []int{2, 3},
            Steps: 2500,
        },
    ))
}

```

Решение + тесты

```
package main
```

```
import (
    "fmt"
    "reflect"
    "sort"
)
```

```
type Statistic struct {
    UserID int
    Steps  int
}
```

```
}
```

```
type Result struct {
```

```
    UserIDs []int
```

```
    Steps int
```

```
}
```

```
func getChampions(statistics [][]Statistic) Result {
```

```
    if len(statistics) == 0 {
```

```
        return Result{}
```

```
    }
```

```
    type UserStatistic struct {
```

```
        DaysIn int
```

```
        StepsSum int
```

```
    }
```

```
    possibleChampions := make(map[int]UserStatistic, len(statistics[0]))
```

```
    for _, userStat := range statistics[0] {
```

```
        possibleChampions[userStat.UserID] = UserStatistic{
```

```
            DaysIn: 1,
```

```
            StepsSum: userStat.Steps,
```

```
        }
```

```
    }
```

```
    for day := 1; day < len(statistics); day++ {
```

```
        for _, userDayStatistic := range statistics[day] {
```

```
            userID := userDayStatistic.UserID
```

```
            if dayStatistic, found := possibleChampions[userID]; found {
```

```
                dayStatistic.DaysIn++
```

```
                dayStatistic.StepsSum += userDayStatistic.Steps
```

```
                possibleChampions[userID] = dayStatistic
```

```
            }
```

```
        }
```

```
    }
```

```
    maxSteps := -1
```

```
    for _, stat := range possibleChampions {
```

```
        if stat.DaysIn == len(statistics) {
```

```
            maxSteps = max(maxSteps, stat.StepsSum)
```

```
        }
```

```
    }
```

```

winners := make([]int, 0, len(possibleChampions))
for userID, stat := range possibleChampions {
    if stat.StepsSum == maxSteps && stat.DaysIn == len(statistics) {
        winners = append(winners, userID)
    }
}
if len(winners) == 0 {
    return Result{}
}

return Result{
    UserIDs: winners,
    Steps:   maxSteps,
}

```

```

func main() {
    fmt.Println("empty", reflect.DeepEqual(getChampions(nil), Result{}))
    fmt.Println("case 1", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 2000}, {UserID: 2, Steps: 1500}},
                {{UserID: 2, Steps: 1000}},
            },
        ),
        Result{
            UserIDs: []int{2},
            Steps:   2500,
        },
    ))
    fmt.Println("case 2", reflect.DeepEqual(
        getChampions(
            [][]Statistic{
                {{UserID: 1, Steps: 2000}, {UserID: 2, Steps: 1500}},
                {{UserID: 2, Steps: 1000}, {UserID: 3, Steps: 4000}},
            },
        ),
        Result{
            UserIDs: []int{2},
            Steps:   2500,
        },
    ))
    fmt.Println("case 3", reflect.DeepEqual(
        getChampions(

```

```

        []Statistic{
            {{UserID: 1, Steps: 2000}, {UserID: 2, Steps: 1500}},
            {{UserID: 2, Steps: 1000}, {UserID: 1, Steps: 3000}},
        },
    ),
    Result{
        UserIDs: []int{1},
        Steps: 5000,
    },
))

// А тут у нас может быть флакующий тест, и так как мы ничего не импортируем
нормального, то напишем так:
case4Answer := getChampions(
    []Statistic{
        {{UserID: 1, Steps: 1000}, {UserID: 2, Steps: 1500}, {UserID: 3, Steps: 1500}},
        {{UserID: 2, Steps: 1000}, {UserID: 1, Steps: 1000}, {UserID: 3, Steps: 1000}},
    },
)
sort.Ints(case4Answer.UserIDs)
fmt.Println("case 4", reflect.DeepEqual(
    case4Answer,
    Result{
        UserIDs: []int{2, 3},
        Steps: 2500,
    },
))
fmt.Println("case 5", reflect.DeepEqual(
    getChampions(
        []Statistic{
            {{UserID: 1, Steps: 2000}, {UserID: 2, Steps: 1500}},
            {{UserID: 2, Steps: 1000}},
            {{UserID: 3, Steps: 1000}},
        },
    ),
    Result{},
))
fmt.Println("case 6", reflect.DeepEqual(
    getChampions(
        []Statistic{
            {{UserID: 1, Steps: 5000}, {UserID: 2, Steps: 1500}},
            {{UserID: 2, Steps: 1000}},
        },
    ),
    Result{

```

```

        UserIDs: []int{2},
        Steps: 2500,
    },
))
}

```

JavaScript

Решение

```

/**
 *
 * @param {Array<Array<{userId: number; steps: number;}>>} statistics
 * @returns {{userIds: number[]; steps: number;} | null}
 */
function getChampions(statistics = []) {
    // Только те, кто был с самого начала, могут победить
    const possibleChampions = statistics[0].reduce((acc, next) => {
        acc[next.userId] = {
            daysIn: 1,
            steps: next.steps
        };
        return acc;
    }, {});

    for (let i = 1; i < statistics.length; i++) {
        for (const result of statistics[i]) {
            if (possibleChampions.hasOwnProperty(result.userId) &&
                possibleChampions[result.userId].daysIn === i) {
                possibleChampions[result.userId].steps += result.steps;
                possibleChampions[result.userId].daysIn += 1;
            }
        }
    }

    // Берем только тех, кто каждый день участвовал
    const activeChampions = Object.entries(possibleChampions).filter(([_, { daysIn }]) => daysIn ===
        statistics.length);
    const maxSteps = Math.max(...activeChampions.map(([_, { steps }]) => steps));

    if (maxSteps === Number.NEGATIVE_INFINITY) {
        return null;
    }

    const winners = {

```



```

    userIds: [],
    steps: maxSteps
  };

  for (let champion of activeChampions) {
    const [userId, {steps}] = champion
    if (steps === maxSteps) {
      winners.userIds.push(+userId);
    }
  }
}

```

Тесты

```

console.log(getChampions(
  [
    [{ userId: 1, steps: 2000 }, { userId: 2, steps: 1500 }],
    [{ userId: 2, steps: 1000 }]
  ]
));

```

// expected: { userIds: [2], steps: 2500 }

```

console.log(getChampions(
  [
    [{ userId: 1, steps: 2000 }, { userId: 2, steps: 1500 }],
    [{ userId: 2, steps: 1000 }, { userId: 3, steps: 4000 }]
  ]
));

```

// expected: { userIds: [2], steps: 2500 }

```

console.log(getChampions(
  [
    [{ userId: 1, steps: 2000 }, { userId: 2, steps: 1500 }],
    [{ userId: 2, steps: 1000 }, { userId: 1, steps: 3000 }]
  ]
));

```

// expected: { userIds: [1], steps: 5000 }

```

console.log(getChampions(
  [

```

```

    [{ userId: 1, steps: 500 }, { userId: 2, steps: 1500 }, { userId: 3, steps: 1000 }],
    [{ userId: 2, steps: 1000 }, { userId: 1, steps: 2000 }, { userId: 3, steps: 1000 }]
  ]
});

// expected: { userIds: [1, 2], steps: 2500 }

console.log(getChampions(
  [
    [{ userId: 1, steps: 500 }, { userId: 2, steps: 1500 }, { userId: 3, steps: 1000 }, { userId: 4, steps:
2500 }, { userId: 5, steps: 12500 }],
    [{ userId: 2, steps: 1000 }, { userId: 1, steps: 2000 }, { userId: 3, steps: 1000 }]
  ]
));
// expected: { userIds: [1, 2], steps: 2500 }

console.log(getChampions(
  [
    [{ userId: 1, steps: 500 }, { userId: 2, steps: 1500 }],
    [{ userId: 2, steps: 1000 }, { userId: 3, steps: 1000 }],
    [{ userId: 3, steps: 1000 }]
  ]
));
// expected: null

```

Kotlin

Решение

```

data class UserStatistics(var steps: Int = 0, var amountDay: Int = 0)
class DayStatistics(val userId: Int, val steps: Int)
class Winners(val userIds: IntArray = intArrayOf(), val steps: Int = 0)

fun getChampions(statistics: Array<Array<DayStatistics>>): Winners {
  if (statistics.isEmpty()) {
    return Winners()
  }
  val map = HashMap<Int, UserStatistics>()
  for (i in statistics.indices) { //аналог i >= 0 && i < statistics.size
    for (j in statistics[i].indices) {

      val dayStatistics = statistics[i][j]
      if (map[dayStatistics.userId] == null && i == 0) {

```

```

        map[dayStatistics.userId] = UserStatistics()
    }
    map[dayStatistics.userId]?.let {
        if (it.amountDay != i) {
            map.remove(dayStatistics.userId)
        } else {
            it.steps += dayStatistics.steps
            it.amountDay++
        }
    }
}
}
}
}
var maxSteps = 0
var resUserIds = ArrayList<Int>()
for (user in map.entries) {
    if (user.value.amountDay == statistics.size) {
        if (maxSteps == user.value.steps) {
            resUserIds.add(user.key)

        } else if (maxSteps < user.value.steps) {
            resUserIds = arrayListOf(user.key)
            maxSteps = user.value.steps
        }
    }
}

return Winners(resUserIds.toIntArray(), maxSteps)
}

```

Тесты

```

fun test(message: String, input: Array<Array<DayStatistics>>, output: Winners) {
    val result = getChampions(input)
    println(
        "$message, ${
            output.userIds.contentEquals(result.userIds) && output.steps == result.steps
        }"
    )
}

test(
    "sample1", arrayOf(
        arrayOf(
            DayStatistics(1, 2000),

```

```
        DayStatistics(2, 1500)
    ),
    arrayOf(DayStatistics(2, 1000))
), Winners(intArrayOf(2), 2500)
)
```

```
test(
    "sample2", arrayOf(
        arrayOf(
            DayStatistics(1, 2000),
            DayStatistics(2, 1500)
        ),
        arrayOf(
            DayStatistics(2, 1000),
            DayStatistics(3, 4000)
        )
    ), Winners(intArrayOf(2), 2500)
)
```

```
test(
    "sample3", arrayOf(
        arrayOf(
            DayStatistics(1, 2000),
            DayStatistics(2, 1500)
        ),
        arrayOf(
            DayStatistics(2, 1000),
            DayStatistics(1, 3000)
        )
    ), Winners(intArrayOf(1), 5000)
)
```

```
test(
    "sample4", arrayOf(
        arrayOf(
            DayStatistics(1, 500),
            DayStatistics(2, 1500),
            DayStatistics(3, 1000)
        ),
        arrayOf(
            DayStatistics(2, 1000),
            DayStatistics(1, 2000),
            DayStatistics(3, 1000)
        )
    )
)
```

```

    ), Winners(intArrayOf(1, 2), 2500)
)

test(
    "sample5", arrayOf(
        arrayOf(
            DayStatistics(1, 500),
            DayStatistics(2, 1500)
        ),
        arrayOf(
            DayStatistics(2, 1000),
            DayStatistics(3, 1000)
        ),
        arrayOf(
            DayStatistics(3, 1000)
        )
    ), Winners(intArrayOf(), 0)
)

```

Swift

Решение

```

struct UserStat {
    let userId: Int
    let steps: Int
}

typealias DailyStats = [UserStat]

struct Champions {
    let userIds: [Int]
    let totalSteps: Int
}

func getChampions(_ stats: [DailyStats]) -> Champions? {
    struct Competitor {
        var totalSteps: Int
        var daysIn: Int
    }

    var daysPassed = 1
    var competitors = [Int: Competitor]()

    stats.first?.forEach { userStat in

```

```

        competitors[userStat.userId] = Competitor(
            totalSteps: userStat.steps,
            daysIn: daysPassed
        )
    }

    guard !competitors.isEmpty
    else { return nil }

    for dayStat in stats.dropFirst() {
        daysPassed += 1
        for userStat in dayStat {
            if let competitor = competitors[userStat.userId] {
                competitors[userStat.userId] = Competitor(
                    totalSteps: competitor.totalSteps + userStat.steps,
                    daysIn: daysPassed
                )
            }
        }
    }

    // Filtering out competitors who missed the day
    competitors = competitors.filter { competitor in
        competitor.value.daysIn == daysPassed
    }

    guard !competitors.isEmpty
    else { return nil }

    var maxSteps = 0
    var winners = [Int]()

    for competitor in competitors {
        if competitor.value.totalSteps > maxSteps {
            winners = [competitor.key]
            maxSteps = competitor.value.totalSteps
        } else if competitor.value.totalSteps == maxSteps {
            winners.append(competitor.key)
        }
    }

    return Champions(userIds: winners.sorted(), totalSteps: maxSteps)
}

```

Решение с использованием редьюсеров

```
struct UserStat {  
    let userId: Int  
    let steps: Int  
}
```

```
typealias DailyStats = [UserStat]
```

```
struct Champions {  
    let userIds: [Int]  
    let totalSteps: Int  
}
```

```
func getChampions(_ stats: [DailyStats]) -> Champions? {  
    struct Competitor {  
        var totalSteps: Int  
        var daysIn: Int  
    }
```

```
    let firstDayCompetitors = stats.first?.reduce(into: [Int: Competitor]()) { competitors, userStat in  
        competitors[userStat.userId] = Competitor(  
            totalSteps: userStat.steps,  
            daysIn: 1  
        )  
    }
```

```
    guard let firstDayCompetitors, !firstDayCompetitors.isEmpty  
    else { return nil }
```

```
    let competitors = stats.dropFirst().reduce(into: firstDayCompetitors) { competitors, dayStat in  
        dayStat.forEach { userStat in  
            if let competitor = competitors[userStat.userId] {  
                competitors[userStat.userId] = Competitor(  
                    totalSteps: competitor.totalSteps + userStat.steps,  
                    daysIn: competitor.daysIn + 1  
                )  
            }  
        }  
    }
```

```
    // Filtering out competitors who missed some days  
    let totalDays = stats.count
```

```

    let allDaysCompetitors = competitors.filter { competitor in
        competitor.value.daysIn == totalDays
    }

    guard !allDaysCompetitors.isEmpty
    else { return nil }

    let maxSteps = allDaysCompetitors.reduce(-1) { maxSteps, competitor in
        max(maxSteps, competitor.value.totalSteps)
    }

    let winners = allDaysCompetitors.compactMap { userId, competitor in
        competitor.totalSteps == maxSteps ? userId : nil
    }

    return Champions(userIds: winners.sorted(), totalSteps: maxSteps)
}

```

Тесты

```

extension Champions: Equatable {}

func test(_ message: String, _ input: [DailyStats], _ output: Champions?) {
    let success = getChampions(input) == output
    print("\(message) > \(success ? "success" : "failed")")
}

test("sample1", [],
    nil
)
test("sample2", [
    [UserStat(userId: 1, steps: 2000), UserStat(userId: 2, steps: 1500)],
    [UserStat(userId: 2, steps: 1000)]
],
    Champions(userIds: [2], totalSteps: 2500)
)
test("sample3", [
    [UserStat(userId: 1, steps: 2000), UserStat(userId: 2, steps: 1500)],
    [UserStat(userId: 2, steps: 1000), UserStat(userId: 3, steps: 4000)]
],
    Champions(userIds: [2], totalSteps: 2500)
)
test("sample4", [
    [UserStat(userId: 1, steps: 2000), UserStat(userId: 2, steps: 1500)],

```



```

    [UserStat(userId: 2, steps: 1000), UserStat(userId: 1, steps: 3000)]
  ],
  Champions(userIds: [1], totalSteps: 5000)
)
test("sample5", [
  [UserStat(userId: 1, steps: 500), UserStat(userId: 2, steps: 1500), UserStat(userId: 3, steps: 1000)],
  [UserStat(userId: 2, steps: 1000), UserStat(userId: 1, steps: 2000), UserStat(userId: 3, steps: 1000)]
],
  Champions(userIds: [1,2], totalSteps: 2500)
)
test("sample6", [
  [UserStat(userId: 1, steps: 500), UserStat(userId: 2, steps: 1500)],
  [UserStat(userId: 2, steps: 1000), UserStat(userId: 3, steps: 1000)],
  [UserStat(userId: 3, steps: 1000)]
],
  nil
)

```

Суммарная неудовлетворённость покупателей (medium)

Условие задачи

На Авито размещено множество товаров, каждый из которых представлен числом. У каждого покупателя есть потребность в товаре, также выраженная числом. Если точного товара нет, покупатель выбирает ближайший по значению товар, что вызывает неудовлетворённость (dissatisfaction), равную разнице между его потребностью и купленным товаром. Количество каждого товара не ограничено, и один товар могут купить несколько покупателей. Рассчитайте суммарную неудовлетворённость всех покупателей.

Нужно написать функцию, которая примет на вход два массива: массив товаров и массив потребностей покупателей, вычислит сумму неудовлетворённостей всех покупателей и вернет результат в виде числа.

Пример

```

# Пример
# ВВОД
goods = [8, 3, 5]
buyerNeeds = [5, 6]

```

вывод

`res = 1` # первый покупатель покупает товар 5 и его неудовлетворённость = 0, второй также покупает товар 5 и его неудовлетворённость = 6-5 = 1

Алгоритмическая сложность

Ожидаемая сложность $O(N \log(N))$. Ожидаемое потребление памяти $O(1)$, дополнительной памяти, кроме входных массивов не выделяется.

Бейзлайн по времени $O(N^2)$

Способы решения

- Полный перебор. Для каждой потребности проходим по массиву товаров и выбираем наиболее близкий по значению. Сложность $O(n*m)$, где n - кол-во товаров, m - кол-во покупателей
- Два указателя. Отсортировать оба массива. Пройти синхронно по обоим массивам, для каждой потребности определяя наилучший товар. Для каждой потребности двигаем указатель по товарам вперёд, пока следующий товар для неё лучше или равен текущему. Сложность $O(\max(n, m) \log(\max(n, m)))$. Более точно $n \log(n) + m \log(m)$. Где n - кол-во товаров, m - кол-во покупателей
- Бинпоиск. Отсортировать массив товаров. Пройти массиву потребностей, для каждой потребности находить в товар бинпоиском. Сложность $O(\max(n, m) \log(n))$. Более точно $n \log(n) + m \log(n)$. Где n - кол-во товаров, m - кол-во покупателей

Основные ошибки

Заполняется по мере работы кандидатов с задачей

Пример решения

Golang

Решение 1. Полный перебор

```
func abs(x uint, y uint) uint {  
    if x < y {  
        return y - x  
    }  
    return x - y  
}  
  
func findTotalDissatisfaction(goods []uint, buyerNeeds []uint) uint {
```

```

var totalDissatisfaction uint
for _, buyerNeed := range buyerNeeds {
    minDiff := ^uint(0)
    for _, good := range goods {
        diff := abs(good, buyerNeed)
        if diff == 0 {
            minDiff = 0
            continue
        } else if diff < minDiff {
            minDiff = diff
        }
    }
    totalDissatisfaction += minDiff
}
return totalDissatisfaction
}

```

Решение 2. 2 указателя

```

func abs(x uint, y uint) uint {
    if x < y {
        return y - x
    }
    return x - y
}

func findTotalDissatisfaction(goods []uint, buyerNeeds []uint) uint {
    var totalDissatisfaction uint
    sort.Slice(goods, func(i, j int) bool { return goods[i] < goods[j] })
    sort.Slice(buyerNeeds, func(i, j int) bool { return buyerNeeds[i] < buyerNeeds[j] })
    var i int
    for _, buyerNeed := range buyerNeeds {
        for i < len(goods)-1 && abs(goods[i+1], buyerNeed) <= abs(goods[i], buyerNeed) {
            i += 1
        }
        totalDissatisfaction += abs(goods[i], buyerNeed)
    }
    return totalDissatisfaction
}

```

Решение 3. Бинпоиск

```

func abs(x uint, y uint) uint {
    if x < y {
        return y - x
    }
    return x - y
}

```

```

    }
    return x - y
}

func findTotalDissatisfaction(goods []uint, buyerNeeds []uint) uint {
    var totalDissatisfaction uint
    sort.Slice(goods, func(i, j int) bool { return goods[i] < goods[j] })
    for _, buyerNeed := range buyerNeeds {
        totalDissatisfaction += abs(binSearchClosest(goods, buyerNeed), buyerNeed)
    }
    return totalDissatisfaction
}

func binSearchClosest(array []uint, target uint) uint {
    left := 0
    right := len(array) - 1
    for left+1 < right {
        mid := (left + right) / 2
        if array[mid] < target {
            left = mid
        } else {
            right = mid
        }
    }
    if abs(array[left], target) < abs(array[right], target) {
        return array[left]
    }
    return array[right]
}

```

Тесты

```

fmt.Println("sample1", findTotalDissatisfaction([]uint{8, 3, 5}, []uint{5, 6}) == 1)
fmt.Println("sample2", findTotalDissatisfaction([]uint{5}, []uint{3, 5, 10, 105}) == 107)
fmt.Println("sample3", findTotalDissatisfaction([]uint{25, 10, 20}, []uint{10, 10, 20}) == 0)
fmt.Println("sample4", findTotalDissatisfaction([]uint{5, 2, 1}, []uint{1}) == 0)
fmt.Println("sample5", findTotalDissatisfaction([]uint{50, 60}, []uint{45, 45, 45}) == 15)
fmt.Println("sample6", findTotalDissatisfaction([]uint{5, 7, 5, 6}, []uint{5, 10, 4}) == 4)

```

JavaScript

Решение 1. Полный перебор

```

function findTotalDissatisfaction(goods = [], buyerNeeds = []) {
    let totalDissatisfaction = 0
    for (let buyerNeed of buyerNeeds) {

```

```

    let minDiff = Number.MAX_VALUE;
    for (let good of goods) {
        const diff = Math.abs(good - buyerNeed);
        if (diff === 0) {
            minDiff = 0;
        } else if (diff < minDiff) {
            minDiff = diff;
        }
    }
    totalDissatisfaction += minDiff;
}
return totalDissatisfaction;
}

```

Решение 2. 2 указателя

```

function findTotalDissatisfaction(goods = [], buyerNeeds = []) {
    goods.sort((a,b) => a - b);
    buyerNeeds.sort((a,b) => a - b);
    let totalDissatisfaction = 0;
    i = 0; // указатель на товары

    for (let need of buyerNeeds) {
        while (i < goods.length - 1 && Math.abs(goods[i + 1] - need) <= Math.abs(goods[i] - need)){
            i += 1;
        }
        totalDissatisfaction += Math.abs(goods[i] - need);
    }

    return totalDissatisfaction;
}

```

Решение 3. Бинпоиск

```

function findTotalDissatisfaction(goods = [], buyerNeeds = []) {
    goods.sort((a,b) => a - b);
    let totalDissatisfaction = 0;
    for (let buyerNeed of buyerNeeds) {
        totalDissatisfaction += Math.abs(binSearchClosest(goods, buyerNeed) - buyerNeed);
    }
    return totalDissatisfaction;
}

function binSearchClosest(array = [], target) {

```

```

let left = 0;
let right = array.length - 1;
while (left + 1 < right) {
    const mid = Math.floor((left + right) / 2);
    if (array[mid] < target) {
        left = mid;
    } else {
        right = mid;
    }
}
if (Math.abs(array[left] - target) < Math.abs(array[right] - target)) {
    return array[left];
}
return array[right];
}

```

Тесты

```

function test(message, input, output) {
    const res = findTotalDissatisfaction(input.first, input.second);
    console.log(message, output == res);
}

```

```

function Pair(first, second) {
    return {first, second}
}

```

```

test("sample1", new Pair([8, 3, 5], [5, 6]), 1)
test("sample2", new Pair([5], [3, 5, 10, 105]), 107)
test("sample3", new Pair([25, 10, 20], [10, 10, 20]), 0)
test("sample4", new Pair([5, 2, 1], [1]), 0)
test("sample5", new Pair([50, 60], [45, 45, 45]), 15)
test("sample6", new Pair([5, 7, 5, 6], [5, 10, 4]), 4)

```

Kotlin

Решение 1. Полный перебор

```

fun findTotalDissatisfaction(goods: IntArray, buyerNeeds: IntArray): Int {
    var totalDissatisfaction = 0
    for (buyerNeed in buyerNeeds) {
        var minDiff = Int.MAX_VALUE
        for (good in goods) {
            val diff = abs(good - buyerNeed)
            if (diff == 0) {

```

```

        minDiff = 0
        continue
    } else if (diff < minDiff) {
        minDiff = diff
    }
}
totalDissatisfaction += minDiff
}
return totalDissatisfaction
}

```

Решение 2. 2 указателя

```

fun findTotalDissatisfaction(goods: IntArray, buyerNeeds: IntArray): Int {
    var totalDissatisfaction = 0
    goods.sort()
    buyerNeeds.sort()

    var i = 0
    for (buyerNeed in buyerNeeds) {
        while (i < goods.size - 1 && abs(goods[i + 1] - buyerNeed) <= abs(goods[i] - buyerNeed)) {
            i += 1
        }
        totalDissatisfaction += abs(goods[i] - buyerNeed)
    }
    return totalDissatisfaction
}

```

Решение 3. Бинпоиск

```

fun findTotalDissatisfaction(goods: IntArray, buyerNeeds: IntArray): Int {
    goods.sort()
    var totalDissatisfaction = 0
    for (buyerNeed in buyerNeeds) {
        totalDissatisfaction += abs(binSearchClosest(goods, buyerNeed) - buyerNeed)
    }
    return totalDissatisfaction
}

```

```

fun binSearchClosest(array: IntArray, target: Int): Int {
    var left = 0
    var right = array.size - 1
    while (left + 1 < right) {
        val mid = (left + right) / 2
    }
}

```

```

    if (array[mid] < target) {
        left = mid
    } else {
        right = mid
    }
}
if (abs(array[left] - target) < abs(array[right] - target)) {
    return array[left]
}
return array[right]
}

```

Тесты

```

fun test(message: String, input: Pair<IntArray, IntArray>, output: Int) {
    val res = findTotalDissatisfaction(input.first, input.second)
    println("$message, ${output == res}")
}

```

```

test("sample1", Pair(intArrayOf(8, 3, 5), intArrayOf(5, 6)), 1)
test("sample2", Pair(intArrayOf(5), intArrayOf(3, 5, 10, 105)), 107)
test("sample3", Pair(intArrayOf(25, 10, 20), intArrayOf(10, 10, 20)), 0)
test("sample4", Pair(intArrayOf(5, 2, 1), intArrayOf(1)), 0)
test("sample5", Pair(intArrayOf(50, 60), intArrayOf(45, 45, 45)), 15)
test("sample6", Pair(intArrayOf(5, 7, 5, 6), intArrayOf(5, 10, 4)), 4)

```

Swift

Решение 1. Полный перебор

```

func dissatisfaction(_ goods: [Int], _ buyerNeeds: [Int]) -> Int {
    var dissatisfaction = 0
    for buyerNeed in buyerNeeds {
        var buyerDissatisfaction = Int.max
        for item in goods {
            buyerDissatisfaction = min(abs(item - buyerNeed), buyerDissatisfaction)
            if buyerDissatisfaction == 0 { break }
        }
        dissatisfaction += buyerDissatisfaction
    }
    return dissatisfaction
}

```

Решение 2. Полный перебор, используя редьюсеры


```

func dissatisfaction(_ goods: [Int], _ buyerNeeds: [Int]) -> Int {
    func buyerDissatisfaction(for buyerNeed: Int) -> Int {
        goods.reduce(Int.max) { dissatisfaction, item in
            min(abs(item - buyerNeed), dissatisfaction)
        }
    }

    return buyerNeeds.reduce(0) { dissatisfaction, buyerNeed in
        dissatisfaction + buyerDissatisfaction(for: buyerNeed)
    }
}

```

Решение 3. 2 указателя

```

func dissatisfaction(_ goods: [Int], _ buyerNeeds: [Int]) -> Int {
    let sortedGoods = goods.sorted()
    let sortedNeeds = buyerNeeds.sorted()

    var dissatisfaction = 0
    var i: Int = 0
    for buyerNeed in sortedNeeds {
        while i < sortedGoods.count-1,
            abs(sortedGoods[i+1] - buyerNeed) <= abs(sortedGoods[i] - buyerNeed) {
            i += 1
        }
        dissatisfaction += abs(sortedGoods[i] - buyerNeed)
    }

    return dissatisfaction
}

```

Решение 4. Бинпоиск

```

func dissatisfaction(_ goods: [Int], _ buyerNeeds: [Int]) -> Int {
    func getClosest(to target: Int, in array: [Int]) -> Int {
        var (left, right) = (0, array.count-1)

        while left + 1 < right {
            let mid = (left + right) / 2
            if array[mid] < target {
                left = mid
            } else {
                right = mid
            }
        }
    }
}

```

```

    }

    return abs(array[left] - target) < abs(array[right] - target)
        ? array[left]
        : array[right]
    }

    let sortedGoods = goods.sorted()

    return buyerNeeds.reduce(0) { dissatisfaction, buyerNeed in
        dissatisfaction + abs(getClosest(to: buyerNeed, in: sortedGoods) - buyerNeed)
    }
}

```

Тесты

```

func test(_ message: String, _ input: ([Int], [Int]), _ output: Int) {
    let success = dissatisfaction(input.0, input.1) == output
    print("\(message) > \(success ? "success" : "failed")")
}

```

```

test("sample1", ([8, 3, 5], [5, 6]), 1)
test("sample2", ([5], [3, 5, 10, 105]), 107)
test("sample3", ([25, 10, 20], [10, 10, 20]), 0)
test("sample4", ([5, 2, 1], [1]), 0)
test("sample5", ([50, 60], [45, 45, 45]), 15)
test("sample6", ([5, 7, 5, 6], [5, 10, 4]), 4)

```

Python

Решение 1. Полный перебор

```

def f(goods, buyer_needs):
    total_dissatisfaction = 0

    for need in buyer_needs:
        closest = min(goods, key=lambda x: abs(x - need))
        total_dissatisfaction += abs(need - closest)

    return total_dissatisfaction

```

Решение 2. 2 указателя

```

def f(goods, buyer_needs):
    goods.sort()

```

```

buyer_needs.sort()

total_dissatisfaction = 0
i = 0 # указатель на товары

for need in buyer_needs:
    while i < len(goods) - 1 and abs(goods[i + 1] - need) <= abs(goods[i] - need):
        i += 1
    total_dissatisfaction += abs(goods[i] - need)

return total_dissatisfaction

```

Решение 3. Бинпоиск

```

def binsearch_closest(arr, target):
    """возвращает ближайшее к target из массива arr"""
    left, right = 0, len(arr) - 1

    while left + 1 < right:
        mid = (left + right) // 2
        if arr[mid] < target:
            left = mid
        else:
            right = mid

    if abs(arr[left] - target) < abs(arr[right] - target):
        return arr[left]
    return arr[right]

```

```

def f(goods, buyer_needs):
    goods.sort()
    total_dissatisfaction = 0

    for need in buyer_needs:
        total_dissatisfaction += abs(need - binsearch_closest(goods, need))

    return total_dissatisfaction

```

Решение 4. Бинпоиск используя стандартную библиотеку

```

import bisect

def f(goods, buyer_needs):

```

```

goods.sort()
total_dissatisfaction = 0

for need in buyer_needs:
    pos = bisect.bisect_left(goods, need)

    if pos == 0:
        closest = goods[0]
    elif pos == len(goods):
        closest = goods[-1]
    else:
        before = goods[pos - 1]
        after = goods[pos]
        if after - need < need - before:
            closest = after
        else:
            closest = before

    total_dissatisfaction += abs(need - closest)

return total_dissatisfaction

```

Тесты

```

assert f([8, 3, 5], [5, 6]) == 1
assert f([5], [3, 5, 10, 105]) == 107
assert f([25, 10, 20], [10, 10, 20]) == 0
assert f([5, 2, 1], [1]) == 0
assert f([50, 60], [45, 45, 45]) == 15
assert f([5, 7, 5, 6], [5, 10, 4]) == 4

```